

MBR WP Performance

Comprehensive User Guide

A practical walkthrough of every tab, setting, and workflow in the plugin — what each option does, when to use it, and when to leave it alone.

Version 1.13.9

Author Made by Robert

Website littlewebshack.com

Support madebyrobert.co.uk

Updated May 2026

Licence GPL v2 or later

Free. No upsells. No premium tiers. No tracking.

Contents

Introduction	5
Who this guide is for	5
How to use this guide	5
What's New in v1.13.x	6
The flagship: bulk AVIF converter and dual-format conversion history	6
The fix that took a while: AVIF false-positive detection	6
The fix that took the longest: "CSS settings reset themselves"	6
Critical CSS chain redesign	7
Three audit fixes	7
UI contrast fix	8
Disable AI Features for WordPress 7.0+	8
HTML Minify edge cases	8
UI polish	8
Getting Started	9
Requirements	9
Installation	9
A safe first-time setup	9
The Admin Interface	11
Tooltips	11
Saving and resetting	11
The thirteen tabs	11
Core Features Tab	12
Scripts & Styles	12
WordPress Features	13
JavaScript Tab	15
CSS Tab	17
Fonts Tab	19
Preloading Tab	21
Lazy Loading Tab	23
Database Tab	25
WebP / AVIF Tab	27

Server Diagnostics	27
Server AVIF capability	27
WebP Conversion Settings	27
AVIF Section	28
Bulk Converter	28
Image Sizing & Dimensions	29
Bulk Resize Tool	29
Server Tab	31
Why this tab matters	31
Server detection	31
Browser Cache Headers	31
Brotli / Gzip Compression	31
On Nginx and IIS	31
Clean removal	32
Third-Party Tab	33
Why self-host tracking scripts	33
Supported scripts	33
How it works	33
Refresh status	33
What this doesn't change	33
Diagnostics Tab	35
Caching Plugin Conflicts	35
Autoloaded Options Audit	35
WP-Cron Viewer	36
Real cron vs. WP-Cron	36
Orphaned Media Tab	37
How detection works	37
Confidence tiers	37
Media types	37
Settings	37
Running a scan	38
The candidate list	38
Deleting orphans	38
Recently Deleted (the restore queue)	39
Daily cron purge	39

A safe first scan	39
WooCommerce Tab	40
Geolocation Advisory	40
Frontend Assets	40
Admin Options	41
Database Cleanup	41
Multisite Network Support	42
Network Admin settings	42
Push to all sites	42
Import from a site	42
Per-site override control	42
New sites	42
Common Workflows	43
"I want to improve my Google PageSpeed Insights score"	43
"My WooCommerce store feels sluggish"	43
"I need to meet GDPR requirements on third-party assets"	43
"My site has ballooned and I want to clean up the database"	44
"I want to delay analytics without breaking them"	44
"I want to reclaim disk space from unused images"	44
Troubleshooting	46
Frequently Asked Questions	49
Resources & Support	52
Where to get help	52
Related plugins in the portfolio	52
Philosophy	52

Introduction

MBR WP Performance is an all-in-one WordPress performance optimisation plugin. Rather than hiding everything behind a handful of presets, it exposes each optimisation as an individual toggle with a clear explanation of what it does and when you would want it on.

Settings are organised across thirteen tabs, each focused on a specific performance area. The admin UI is accessed from the WordPress toolbar — there is no sidebar menu item, which keeps the wp-admin sidebar tidy. The plugin is completely free with no premium tiers, no upsells, and no external tracking.

Who this guide is for

This guide is aimed at WordPress site owners, developers, and agencies running the plugin on live sites. It assumes familiarity with basic WordPress concepts (plugins, themes, the admin area) but does not require deep technical knowledge. Where a particular setting has technical implications, those are called out explicitly in callout boxes.

How to use this guide

Each tab has its own chapter. Within each chapter you will find a short description of what that area of the plugin targets, followed by a walk-through of each setting with guidance on when to enable it. You do not need to read the guide front-to-back — jump to the tab you are configuring and follow that chapter.

If you are setting up the plugin for the first time, start with the Getting Started chapter which walks you through a safe first-time configuration in about ten minutes. If you have upgraded from v1.12.x or earlier, read the next chapter first — it summarises every change in the 1.13.x line and where to find the new behaviour.

Tip

Enable one feature at a time and test your site after each change. Performance plugins can interact in surprising ways with themes, page builders, and other plugins. Individual toggles make it easy to narrow down any issues.

What's New in v1.13.x

The 1.13.x line is a stability and feature-completion run on top of the major v1.12.0 release. If you're upgrading from v1.12.x, the migrations are automatic and your settings are preserved. Skim this chapter to learn where the new controls live and which long-standing bugs have finally been put to bed.

The flagship: bulk AVIF converter and dual-format conversion history

New in v1.13.8. The WebP / AVIF tab now has a Bulk AVIF Converter section alongside the existing WebP one, with its own Start / Clear History / Revert All buttons. Per-image AJAX with a progress bar, registry-driven Revert All that deletes every .avif this plugin created without touching originals or WebP variants. Auto-convert on upload also writes to a parallel AVIF history option, so freshly-uploaded images appear in the table alongside bulk-converted ones.

The Conversion History table itself has been redesigned to merge records from both WebP and AVIF histories, keyed by original image path. A new **AVIF Size** column sits alongside the existing WebP Size column. Each image appears as a single row with whichever format data exists — a dash shows where a format hasn't been generated for that image. The Compression column reports savings against whichever recorded format is smallest (AVIF when present, since it's typically 20–30% smaller than WebP at equivalent perceived quality).

The Bulk AVIF Converter only renders when the server has a real AVIF encoder available — see the next item.

The fix that took a while: AVIF false-positive detection

New in v1.13.7. Server AVIF support was being detected via `function_exists('imageavif')`, which is unreliable. From PHP 8.1 onwards the `imageavif()` function is declared whether or not libgd was actually compiled against libavif; on the very common shared-host configuration where it wasn't, the function exists but calls fail silently at runtime. No .avif files are produced, visitors are served WebP (or the original), and the user sees no error.

Detection now uses `gd_info()['AVIF Support']`, which reflects what libgd was actually built with — the same reliable pattern the plugin's WebP detection has always used via `gd_info()['WebP Support']`.

When AVIF is enabled in settings but the server can't actually encode AVIF, the plugin now does *not* register the upload-conversion filter (so it won't trigger per-upload warnings) and shows an admin notice on its own admin pages explaining the mismatch. The AVIF capability diagnostic on the WebP tab now always renders — both when supported (info notice with a quick confirmation) and when unsupported (warning notice with the GD / Imagick breakdown).

The fix that took the longest: "CSS settings reset themselves"

New in v1.13.5. Three font-related fields on the Fonts tab — Optimise Google Fonts, Font Display Strategy (duplicate), and Disable Elementor Google Fonts — had their HTML form inputs scoped to `mbr_wp_performance_options[css][...]` instead of `[fonts][...]`. Submitting the Fonts tab therefore POSTed a partial css section, and the CSS sanitiser — doing exactly what a section sanitiser should do — replaced the entire stored css section with the partial input, setting every CSS boolean not

present in the form to false and dropping the textareas.

This is what users had been reporting as *"the CSS settings all return to default (switched off) without warning"* after some unknown period of time. The unknown period was the time between saving the CSS tab and saving the Fonts tab. The trigger was reliably "click Save on the Fonts tab."

The two dead-UI fields (the radio with no runtime consumer, and the duplicate Font Display Strategy select) have been removed. The Disable Elementor Google Fonts checkbox has moved to its correct home on the Fonts tab, with a migration that preserves any existing value. The plugin no longer has any cross-section form fields anywhere — each tab's fields are now strictly scoped to that tab's section.

Critical CSS chain redesign

New in v1.13.4. The Auto-Generate Critical CSS button has been removed. Its regex-based extraction across hardcoded selectors was structurally too crude to produce safe inline CSS for modern WordPress sites — it stripped @media wrappers off rules, matched anything starting with a single-letter target like p or a, ignored CSS variables defined on :root, and indiscriminately inlined every @font-face declaration on the site. The output of inlining incomplete or broken critical CSS while async-loading the rest produced a flash of unstyled (or incorrectly styled) content that several users reported as "the optimisations have stopped working" — when in fact the toggle states were intact and only the visual result was wrong.

The Critical CSS Code textarea remains. The recommended workflow is now to paste critical CSS produced by a proper viewport-aware tool — Penthouse, Critical, Critters, or any of the online critical-CSS generators — into the textarea, enable Inline Critical CSS, and save. See the CSS Tab chapter for the full workflow.

To make Async CSS safer when no critical-CSS bridge is in place, a safety interlock has been added. If "Load CSS Asynchronously" is enabled *without* a critical-CSS bridge (Inline Critical CSS on + Critical CSS Code populated), the first two render-blocking-eligible stylesheets stay render-blocking and the rest are async'd. This guarantees the page paints with real CSS at first paint regardless of how the rest of the chain is configured. With a critical-CSS bridge in place, every stylesheet is async'd as before.

Three audit fixes

New in v1.13.3.

- **Critical CSS persistence:** the previous Auto-Generate button stored its output under an internal key (`[css][critical_css_content]`) that wasn't in the CSS sanitiser's allow-list, so the sanitiser stripped it on every save. Generated CSS only survived at all because the admin JS dropped it into the editable textarea too. With the generator now removed (above), this is moot — but the underlying field plumbing has been cleaned up.
- **Reset to Defaults button:** was inert. The button navigated to `?...&reset=1` but no handler ever read that parameter. It now runs a proper nonce- and capability-checked POST request that resets every section to its defaults and reloads the page. Implemented as POST rather than a GET link so it can't be triggered by a crawler, prefetch, or stray bookmark.
- **Autoload Audit SQL:** the Diagnostics → Autoloaded Options audit claimed in its docblock to exclude the plugin's own options but the SQL didn't, so MBR WP Performance's own options could appear in the list and be offered for autoload-disabling. The query now excludes the `mbr_wp_performance_` namespace as documented.

UI contrast fix

New in v1.13.6. Inline status messages rendered by AJAX handlers — visible on the Database panel and elsewhere — appeared with WordPress core's near-black `.notice` text colour on the plugin's dark-themed success / error / warning / info backgrounds, producing low-contrast pills that were hard to read against the green / red / amber tints. The plugin's `.notice-*` rules now set the lighter primary text colour and apply it to nested `<p>` elements too, so the readable text wins over any descendant rules WP core may inject.

Disable AI Features for WordPress 7.0+

New in v1.13.0. The headline feature of the 1.13.0 release proper, on the Core Features tab under WordPress Features. WordPress 7.0 ships a built-in AI subsystem — an AI Client, the Abilities API, and a Settings → Connectors screen for wiring the site to AI providers. It stays dormant until a provider connector is configured. This toggle switches the whole subsystem off via core's own kill switch (`wp_supports_ai` → `__return_false` at `PHP_INT_MAX`), with `wp_ai_client_prevent_prompt` as a second guard.

Off by default. No effect on WordPress 6.x (the filter doesn't exist there), so it's safe across mixed-version sites. The Connectors admin screen is intentionally left in place — this is a performance and surface-area control, not a dashboard-hiding tool.

HTML Minify edge cases

Two patches to Minify HTML reached the 1.13.x line via *v1.13.1* and *v1.13.2*. The first carried forward fixes from 1.12.x: a critical placeholder bug where the protection token used during minification was itself an HTML comment and got stripped by the comment-removal pass (taking inline scripts and styles with it), plus more conservative whitespace handling that preserves inline-element spacing and `<svg>` content. The second handled pages that embed a complete HTML document inside a page-builder HTML widget (e.g. an Elementor "HTML" widget holding a full `<!doctype html>...</html>` landing page) — those pages are now detected and byte-for-byte preserved, because collapsing whitespace across a nested document can confuse the browser's parser.

UI polish

New in v1.13.9. The "Compression" column header on the Conversion History table was 110px wide, just narrow enough that the word wrapped onto a second line at the standard wp-list-table header font weight. Bumped to 140px so the label sits cleanly on one line.

Note

If you previously enabled Combine JavaScript, Combine CSS, or Remove Unused CSS, the UI toggles are preserved for forward compatibility but currently act as no-ops with a clear admin notice. A safe implementation of each requires substantial engineering work scheduled for a future release. Use Defer, Delay, Inline Critical CSS, and Async CSS in the meantime — they give similar Lighthouse wins without the risk.

Getting Started

Requirements

- WordPress 5.8 or higher (tested up to 7.0)
- PHP 7.4 or higher (PHP 8.1 or higher with libavif for the GD AVIF path)
- MySQL 5.6 or higher
- GD library with WebP support (for image conversion and resizing)
- *Optional:* Imagick 7.0.25+ with libheif, or GD with libavif compiled in — for AVIF conversion. Capability is auto-detected via `gd_info()[ 'AVIF Support ' ]` for the GD path; the AVIF section is hidden in the UI if neither is present.

Installation

Download the latest release zip from littlewebshack.com or the plugin's GitHub releases page, then install it via Plugins → Add New → Upload Plugin. Upload the zip, click Install Now, then Activate. You can also extract the zip and upload the resulting folder to `/wp-content/plugins/` via SFTP.

After activation, look for **WP Performance** in the WordPress admin toolbar at the top of the screen. Clicking it takes you straight to the settings page.

A safe first-time setup

If you are just getting started and want a sensible baseline that will not break anything, enable the following settings in this order. Test your site after each step before moving on.

- **Diagnostics tab:** Open this first. Check the Caching Plugin Conflicts panel — if any caching plugin is detected, work out which optimisations belong to which plugin before enabling anything. Have a quick look at the Autoloaded Options Audit and the WP-Cron Viewer for any obvious red flags.
- **Database tab:** Run a post revisions cleanup (keep the last 5), delete expired transients, and run the orphaned metadata scan. Click Run Auto-Cleanup Now once to clear any backlog.
- **Server tab:** Enable Browser Cache Headers and Brotli / Gzip Compression. Both write to `.htaccess` on Apache and LiteSpeed; on Nginx you will see a copy-ready snippet to paste into your server config.
- **Lazy Loading tab:** Enable Lazy Load Images and Lazy Load iFrames. If you embed YouTube or Vimeo videos in posts, enable the YouTube / Vimeo Facade too — it is a substantial win.
- **Core Features tab:** Enable Disable Emojis, Disable Embeds (only if you do not use them), Remove Query Strings, and Minify HTML. If you're on WordPress 7.0 and have no plans to wire the site to an AI provider, also enable Disable AI Features.
- **WebP / AVIF tab:** Check the Server Diagnostics panel. If WebP is supported, enable Automatic Conversion and run the WebP bulk converter on your existing Media Library. If AVIF is also supported, run the AVIF bulk converter afterwards — same picture-tag delivery, smaller files.
- **WebP / AVIF tab → Image Sizing & Dimensions:** Enable Resize Large Uploads (default 2560px), Add Missing Width & Height, `decoding="async"`, and Strip EXIF Metadata.
- **Preloading tab:** Enable Hover Prefetch. Auto-prioritise First Image too if your hero is reliably the LCP element.

- **Third-Party tab:** Enable self-hosting for whichever tracking scripts you use. Click Refresh Now to populate the cache.
- **WooCommerce tab** (*if applicable*): Disable cart fragments on non-shop pages, enable Disable WC Scripts on Non-Shop Pages, and set Action Scheduler retention to 14 days.

Note

The plugin automatically disables all optimisations inside page builder editors (Elementor, Beaver Builder, Divi, Oxygen, Bricks, WPBakery) to prevent conflicts with the editing experience. You do not need to configure this — detection is automatic. Upload-pipeline modules (WebP, AVIF, EXIF strip, resize) still run regardless of editor context as of v1.12.2, so images uploaded through builder media pickers get the same treatment as direct Media Library uploads.

The Admin Interface

The settings page is accessed from the **WP Performance** item in the admin toolbar. Each tab corresponds to a specific area of optimisation, and all tabs share the same layout: a list of settings with tooltips, a Save Changes button at the bottom, and a Reset to Defaults button.

Tooltips

Every setting has a question-mark icon next to its label. Hovering over it reveals a tooltip with a brief explanation of what the setting does. Tooltips are the fastest way to understand a setting without leaving the admin.

Saving and resetting

The Save Changes button saves the settings on the current tab. Settings on other tabs are not affected. The Reset to Defaults button (now functional from v1.13.3 onwards — see What's New) resets every setting across every tab. It runs a nonce-protected POST request and reloads the page, so it cannot be triggered accidentally by a crawler or prefetch. Use it carefully.

The thirteen tabs

Tab	Focus area
Core Features	WordPress-level features: emojis, embeds, XML-RPC, REST API, heartbeat, HTML minify, disable WordPress 7.0 AI
JavaScript	Defer, jQuery, minification, delay until interaction
CSS	Inline critical CSS, async loading with safety interlock, minification, unused styles
Fonts	Google Fonts self-hosting, preloading, font-display, Font Awesome, Disable Elementor Google Fonts
Preloading	Image preload, fetch priority, speculative loading, hover prefetch
Lazy Loading	Native image / iFrame lazy loading, YouTube / Vimeo facade
Database	Revisions, transients, orphaned metadata, table optimisation, scheduled cleanup
WebP / AVIF	WebP + AVIF conversion, bulk converters for both, image resizing, decoding=async, EXIF strip
Server	Browser cache headers, text compression (Brotli / Gzip)
Third-Party	Self-host Google Analytics, GTM, Facebook Pixel
Diagnostics	Autoload audit, WP-Cron viewer, caching plugin conflict detector
Orphaned Media	Detect and safely remove unreferenced media attachments
WooCommerce	WooCommerce-specific optimisations

Core Features Tab

The Core Features tab groups WordPress-level toggles that disable default behaviour you may not need. Most of these are safe to enable. A small number require a bit of judgement about what your site actually uses.

Scripts & Styles

Disable Emojis

WordPress injects a JavaScript detection script and inline CSS on every page to support emoji rendering on older browsers. Modern browsers render native emojis without this help. Safe to enable unless you specifically use WordPress's emoji conversion on very old mobile browsers.

Disable Dashicons on Frontend

Dashicons is WordPress's admin icon font. It is loaded on the frontend for logged-in users and sometimes for all visitors depending on the theme. Only enable if you are certain your theme does not use Dashicons in its public-facing templates. Check by temporarily enabling it on a staging copy and inspecting your site while logged out.

Disable Embeds

Removes the oEmbed functionality that converts pasted URLs (YouTube, Twitter, etc.) into rich embeds, along with the embed detection script. If you embed external content in posts, leave this off. If your site is purely text and images, it is a useful win.

Remove jQuery Migrate

jQuery Migrate is a compatibility layer that lets older plugins keep working with newer versions of jQuery. Modern, actively maintained plugins rarely need it. Test carefully after enabling — if any plugin breaks, disable this first.

Remove Global Styles

Block themes and Full Site Editing inject a large chunk of inline CSS on every page to handle theme.json design tokens. If you are using a classic (non-block) theme, this CSS is dead weight and can be removed safely. Auto-skipped on block themes since v1.9.2.

Minify HTML

Output-buffers your front-end HTML response and strips HTML comments and redundant whitespace before delivery. The contents of `<pre>`, `<textarea>`, `<script>`, `<style>`, and `<svg>` are carefully preserved via a collision-free per-request token, IE-style conditional comments are kept, and pages embedding a nested complete HTML document inside a builder HTML widget are skipped automatically. AMP, REST, and AJAX responses are also skipped. Typically saves 5–15% of HTML transfer size with no visible effect.

Note

With Brotli / Gzip compression enabled (Server tab), HTML minification saves very little of the *compressed* transfer — it's the lowest-impact optimisation here, so don't feel obliged to use it.

WordPress Features

Disable XML-RPC

XML-RPC is a remote API that predates the REST API. It is mainly used by the old WordPress mobile apps and Jetpack. If you do not use either, disabling XML-RPC reduces attack surface and blocks a common brute-force vector.

Hide WordPress Version

Removes the WordPress version meta tag and generator strings from the frontend. Security through obscurity only — it will not stop a determined attacker who can fingerprint your site in other ways, but there is no downside to enabling it.

Disable RSS Feeds

If your site does not publish content that anyone subscribes to via RSS, this removes the feed URLs and prevents them being scraped. Most brochure sites and e-commerce stores can safely disable feeds. Blogs should leave them on.

Disable Self-Pingbacks

Stops WordPress notifying itself when one post links to another on the same site. Self-pingbacks generate useless comments and extra database activity. Safe to enable.

REST API Control

Three modes are available. **Default** leaves the REST API fully open as WordPress ships it. **Logged-in Only** restricts the API to authenticated users, which breaks any frontend feature that uses the API for anonymous visitors — block editor previews, some form plugins, and most headless setups. **Block Public User Enumeration** is the middle ground: the API stays open for most endpoints but the `/users` endpoint no longer reveals usernames to unauthenticated requests. This is usually the right choice for brochure sites and e-commerce stores.

An allowlist of REST namespaces is available when either of the non-default modes is selected — admins can whitelist specific namespaces (e.g. `mbr-isa/v1`, `contact-form-7/v1`, `wc/store/v1`) that should remain accessible to anonymous visitors.

Heartbeat API

The Heartbeat API is WordPress's real-time communication layer used by autosave, post locking, and various plugins. It fires an admin-ajax request every 15 seconds by default, which can generate significant server load on busy sites.

Options are Default, Disable on Dashboard, Disable Everywhere, and a custom frequency setting (in seconds). For most sites, setting the frequency to 60 seconds is a good balance between responsiveness and server load.

Post Revisions & Autosave

Controls how many post revisions WordPress stores and how often it autosaves drafts. Reducing revisions to 5 or 10 keeps the posts table lean without losing too much edit history. Autosave every 60 seconds is WordPress's default and rarely needs changing.

Remove Query Strings

Some older caching proxies refuse to cache URLs that contain query strings like `?ver=1.2.3` on CSS and JS files. Removing them improves cacheability. Modern page cache plugins handle this correctly regardless, but the toggle is harmless.

Disable AI Features (WordPress 7.0+)

New in v1.13.0. WordPress 7.0 ships a built-in AI subsystem — an AI Client, the Abilities API, and a Settings → Connectors screen for wiring the site to AI providers. It stays dormant until a provider connector is configured, so the default-install cost is minimal. This toggle switches the whole subsystem off via core's own kill switch (`wp_supports_ai` → `__return_false` at `PHP_INT_MAX`), with `wp_ai_client_prevent_prompt` as a second guard.

Off by default. No effect on WordPress 6.x (the filter doesn't exist there), so it's safe across mixed-version sites. The Connectors admin screen is intentionally left in place — this is a performance and surface-area control, not a dashboard-hiding tool. Enable when you have no intention of connecting AI providers and want the subsystem out of the bootstrap path entirely.

JavaScript Tab

The JavaScript tab controls how and when JS files load on your site. Used thoughtfully, the options here can produce large improvements in Time to Interactive and First Contentful Paint scores. Used carelessly, they can break your site. Test after every change.

Defer JavaScript

Adds the `defer` attribute to script tags, which tells the browser to download the script in parallel with HTML parsing but only execute it after the HTML is fully parsed. This prevents render-blocking but preserves execution order.

Almost all scripts can safely use `defer`. The main exceptions are scripts that the theme relies on for inline execution during page render — but those are increasingly rare. Use the Defer exclusion list for any individual script that needs to keep its default loading behaviour.

Defer jQuery

A narrower version of Defer JavaScript that only applies to the `jquery`, `jquery-core` and `jquery-migrate` handles. Useful when you want most scripts to run normally but jQuery itself is render-blocking.

Move Scripts to Footer

Hoists `<script>` tags out of the `<head>` and into the footer. Render-blocking JS in the head delays the first paint; moving it to the footer means the user sees content sooner. Combine with Defer for the biggest effect. Scripts listed in the exclusion textarea stay in the head.

Remove jQuery

Warning

This is the most disruptive toggle in the plugin. An enormous amount of WordPress code — themes, plugins, page builders — depends on jQuery being available globally. Enable only if you know your theme and every active plugin ships without jQuery dependencies. Test extensively on a staging copy first.

Test mode (in the same section) leaves jQuery in place for logged-in users so you can compare behaviour live without breaking the front-end for visitors. Once you are confident, disable test mode for the optimisation to apply site-wide.

Minify JavaScript

Strips whitespace and comments from inline scripts in the served page. External script files are left alone — those are typically already minified at build time, and minifying cached assets at runtime is wasteful. Compatible with most themes. If you see a JS error immediately after enabling, disable and report which script broke.

Delay JavaScript Until Interaction

This is a powerful setting. Any script matching the delay list will not execute until the user scrolls, clicks, hovers, or presses a key. This can push analytics, chat widgets, and tag managers entirely off the critical rendering path.

Common candidates for delay: Google Tag Manager, Google Analytics, Facebook Pixel, Hotjar, Intercom, Tawk.to, HubSpot tracking. Use the exclusion list to prevent delays on scripts that must run

immediately (the Cookie Consent plugin's own script, for instance).

A configurable timeout (default 3 seconds) ensures delayed scripts run eventually even if the user never interacts — useful for analytics that would otherwise miss bounced sessions entirely.

Tip

Delay until interaction can dramatically improve Lighthouse scores, but measure the impact on conversion data. Analytics scripts that delay will miss users who bounce before interacting. Some analytics providers accept this trade; others do not. The configurable timeout limits the worst case.

Disable Concatenation

WordPress concatenates admin scripts by default when the user is logged in. On some hosting environments this triggers timeouts during `load_scripts`. Enable to define `CONCATENATE_SCRIPTS = false` for the affected request.

Remove Script Versions

Strips the `?ver=` query string from script URLs. Useful for older caching proxies that refuse to cache URLs with query parameters.

Combine JavaScript

Preserved in the UI for forward compatibility but currently a no-op. Enabling the toggle surfaces an admin notice clarifying that the option is not yet implemented. A safe implementation handling dependency graphs and async order is a substantial separate engineering project. Use `Defer`, `Delay`, or `Move-to-Footer` instead — they give similar Lighthouse wins without the risk.

Exclusion Lists

Each of the above options (`Defer`, `Defer jQuery`, `Move-to-Footer`, `Delay`) has its own exclusion list. Paste one script identifier per line — you can match on filename, handle, path fragment, or inline script content. Exclusions are applied as a simple substring match, so being too loose (say, `js`) will exclude everything.

CSS Tab

The CSS tab manages stylesheet delivery. Used together with proper critical CSS, the options here move stylesheet loading off the critical rendering path so the first paint happens without waiting for full stylesheets to download.

Minify CSS

Strips whitespace and comments from inline style tags in the served page. External stylesheets are left alone. Almost always safe.

Load CSS Asynchronously

Changes CSS loading from render-blocking to non-blocking using the preload + onload pattern. A small loadCSS polyfill is included automatically for older browsers that do not fire the onload event on preload links.

Async-loading every stylesheet without inline critical CSS produces a Flash of Unstyled Content (FOUC) — the page renders briefly with no styles before the CSS arrives. From v1.13.4 the plugin protects against this with a safety interlock described below.

Async-CSS Safety Interlock

New in v1.13.4. When "Load CSS Asynchronously" is enabled *without* a critical-CSS bridge (Inline Critical CSS on + Critical CSS Code populated), the first two render-blocking-eligible stylesheets stay render-blocking and the rest are async'd. This guarantees the page paints with real CSS at first paint regardless of how the rest of the chain is configured. With a critical-CSS bridge in place, every stylesheet is async'd as before, full optimisation.

This interlock removes a long-standing footgun: async-loading every stylesheet without inline critical CSS produces a FOUC while the preloaded links resolve. Most users misread that paint window as "the layout is broken" or "the plugin isn't working" rather than its actual cause. The interlock simply refuses to enter the failure mode.

Inline Critical CSS

Critical CSS is the subset of CSS rules needed to render the initial viewport — the area visible without scrolling. Inlining critical CSS in <head> lets the page paint immediately, while the full stylesheets load asynchronously.

The plugin no longer ships an in-house critical CSS generator. The previous Auto-Generate button used regex-based extraction across a hardcoded selector list (body, header, h1, a, etc.), which is structurally too crude to produce safe inline CSS for a modern theme or page-builder site. It stripped @media wrappers off rules, matched anything starting with a single-letter target, ignored CSS variables defined on :root, and indiscriminately inlined every @font-face declaration on the site. The output frequently broke layouts more than it improved performance.

The recommended workflow is:

1. Generate critical CSS using a proper viewport-aware tool — Penthouse, Critical, Critters, or any of the online critical-CSS generators that render the page in a headless browser at a target viewport and extract only the rules whose selectors match elements above the fold.
2. Paste the output into the Critical CSS Code textarea on the CSS tab.
3. Enable Inline Critical CSS.

4. Save.

Note

If you enable Async CSS without pasting any critical CSS, the safety interlock above keeps the first two stylesheets render-blocking so the page still paints correctly. Providing critical CSS unlocks full async loading and the strongest First Contentful Paint.

CSS Scanner (Unused Styles)

The scanner analyses which CSS rules from your stylesheets are actually used on the homepage. It will not automatically remove unused CSS — that is too risky — but it gives you a report showing which files have the most unused styles, useful for deciding where to focus manual optimisation.

Combine CSS

Preserved in the UI for forward compatibility but currently a no-op, same as the JS counterpart. Async CSS plus Inline Critical CSS together give similar real-world benefits without the risk of cascade-ordering issues.

Remove Unused CSS

Preserved in the UI for forward compatibility but currently a no-op. A safe implementation requires a proper DOM scanner — substantial engineering work. For per-asset CSS control today, the *MBR Advanced Asset Manager* plugin lets you conditionally dequeue stylesheets on a per-URL basis.

Separate Block Styles

Tells WordPress to only load the CSS for blocks that actually appear on the page, rather than loading the full block library stylesheet on every page. Pure win on sites that use the block editor.

Disable WooCommerce CSS

Equivalent of the WooCommerce tab's stylesheet-dequeue toggle, retained here for discoverability. Only enable one or the other.

Remove CSS Versions

Strips the `?ver=` query string from style URLs. Same rationale as the script version remover on the JavaScript tab.

Note

Disable Elementor Google Fonts used to live here. In v1.13.5 it moved to the Fonts tab — its correct home — as part of fixing a long-running bug where saving the Fonts tab could reset CSS toggles. The migration runs automatically; any value you had set is preserved. See the Fonts Tab chapter for the setting itself.

Fonts Tab

Web fonts are one of the most commonly mishandled performance costs on WordPress sites. Every Google Fonts request introduces render-blocking HTTP, DNS resolution, and TLS handshakes against third-party domains. The Fonts tab gives you tools to bring fonts under your own domain's control.

Self-Host Google Fonts

The headline feature. Enable Self-Host Google Fonts and the plugin scans your site for Google Fonts requests, downloads the font files to your server, and rewrites the stylesheet URLs to serve from your own domain. No more calls to `fonts.googleapis.com` or `fonts.gstatic.com`.

Click Download Fonts to perform the initial scan and download. After changing your font stack (in the theme or page builder), click Clear Font Cache and re-download.

Note

Self-hosting Google Fonts also addresses GDPR concerns in the EU. When Google Fonts are served from Google's CDN, visitor IP addresses are transmitted to Google. Self-hosting keeps that data on your server.

Manual Fonts

If your theme declares Google Fonts in a way the automatic scanner does not detect — for example, hard-coded in a PHP template — you can list the font families manually in this field. The plugin downloads them on save. Useful for edge cases where detection fails.

Preload Critical Fonts

Preloading fonts tells the browser to start downloading them as soon as possible, rather than waiting until the CSS that uses them is parsed. Enable Preload Critical Fonts and list the fonts that appear above the fold. Do not preload every font — it hurts more than it helps.

All preload link tags emitted by the plugin use the explicit `crossorigin="anonymous"` value rather than the bare attribute, fixing inconsistent Lighthouse audits.

Font Display Strategy

Controls what the browser does while a web font is still loading. Four options:

- **swap**: Show the fallback font immediately, then swap to the web font when it loads. Best for performance; there will be a visible text swap.
- **block**: Hide text for up to 3 seconds waiting for the font, then fall back. Rarely a good choice — invisible text looks broken.
- **fallback**: Short block period then fallback; if the font arrives quickly, use it, otherwise keep the fallback for the rest of the page.
- **optional**: Like fallback but even more aggressive — if the font is slow, it will not be used at all on the current page load. Best font-display for flaky connections.

Default to **swap** unless you have a specific reason to choose otherwise.

Font Subsetting

When enabled, the plugin strips glyphs from downloaded fonts that are not needed for the character sets you have selected. A Latin-only subset can be 60–80% smaller than the full file. Safe for English

and Western European sites; be careful if your content includes Cyrillic, Greek, or CJK characters.

Font Awesome Optimisation

Font Awesome often arrives as a second, large web font on sites that already bundle icons via their theme. Options are Default (no change), Optimise (load only the subset actually used), and Disable (do not load Font Awesome at all). Use Disable only if you are certain nothing on your site uses Font Awesome icons.

Disable Elementor Google Fonts

Moved here from the CSS tab in v1.13.5. Elementor injects Google Fonts link tags based on its widget configuration. If you have self-hosted your fonts via this tab, this toggle stops the duplicate external requests. Only relevant on Elementor sites — the toggle only appears when `ELEMENTOR_VERSION` is defined.

This setting moved from the CSS tab to the Fonts tab in v1.13.5 as part of fixing the long-running "CSS settings reset themselves" bug. See the What's New chapter for the full story. The migration moves any existing value across automatically; you don't need to do anything on upgrade.

Preloading Tab

Preloading and speculative loading are hints you give the browser about what resources it will need soon. Used judiciously, they reduce perceived latency. Overused, they waste bandwidth and can actually hurt performance. The options here reward careful thought.

Preload Critical Images

Tells the browser to start downloading specific images as soon as it parses the HTML, rather than waiting until they are encountered in the DOM. The most common use case is the Largest Contentful Paint (LCP) image — the hero image at the top of a landing page.

List image URLs or CSS selectors to identify which images should be preloaded. Be specific: preloading images that are below the fold wastes bandwidth and competes with resources the browser actually needs.

Fetch Priority

The `fetchpriority` HTML attribute lets you explicitly tell the browser which images matter most. Three related settings:

- **Auto-prioritise First Image:** Automatically sets `fetchpriority="high"` on the first image found in the main content area. Sensible default for most sites.
- **High Priority Selectors:** CSS selectors for additional critical images that should get high priority. Good for landing pages with multiple above-the-fold images.
- **Disable Core Fetch Priority:** WordPress core added its own `fetchpriority` logic in 6.3, which sometimes conflicts with manual optimisation. Disable if you want complete control.

Preconnect & DNS Prefetch

Two related hints for third-party domains. Preconnect opens a full DNS + TLS connection in advance (heavier — use for one or two critical domains). DNS Prefetch only resolves the DNS (lightweight — fine for many domains). Use Preconnect for domains you fetch from on the critical path (font CDN, image CDN); DNS Prefetch for everything else (analytics, ads, third-party widgets).

Cloudflare Early Hints

HTTP 103 Early Hints let your server tell Cloudflare (or another edge cache) which resources to start preloading even before the origin finishes generating the HTML response. If your site is behind Cloudflare with Early Hints enabled in the dashboard, enabling this toggle tells the plugin to emit the Link headers Cloudflare needs.

Speculative Loading

The Speculation Rules API lets browsers prefetch or prerender the next page the user is likely to visit. When they click a link that has been pre-rendered, the page appears instantly. Four eagerness settings:

- **Conservative:** Only prefetch on explicit mouseover or touchstart.
- **Moderate:** Default. Prefetch links on hover.
- **Eager:** Start prefetching as soon as a link appears in the viewport.
- **Auto:** Let the browser decide.

Warning

Speculative loading consumes server resources even for pages the user never ends up visiting. On high-traffic sites, "Eager" can significantly increase server load. Start with "Moderate" and monitor your server metrics before escalating.

Hover Prefetch (instant.page)

A complementary mechanism to the Speculation Rules API for browsers that don't yet support it. On link hover (or first touchstart on mobile), the destination page is prefetched so the next click feels instant. Uses the canonical instant.page v5.2.0 runtime, which is MIT licensed.

Honours the Save-Data: on request header — the runtime never fires for visitors on metered connections. Server-side bail in PHP avoids even emitting the script in those cases.

Lazy Loading Tab

Lazy loading delays the loading of images and iFrames until they are about to scroll into view. The plugin uses the browser's native `loading="lazy"` attribute where possible, falling back to `IntersectionObserver` for older browsers and for features native lazy loading does not support.

Lazy Load Images

Enables lazy loading for all `<img>` tags on the frontend. Native lazy loading is supported in all modern browsers and produces essentially no CLS when image dimensions are known — pair it with the Add Missing Width & Height option in the WebP / AVIF tab.

Lazy Load iFrames

Same treatment for embedded iFrames (YouTube, Vimeo, Google Maps, Twitter embeds). Often a bigger win than image lazy loading because iFrames bring their own JS payloads. Enable this early.

For YouTube and Vimeo specifically, the **Video Facade** option below is a stronger optimisation — it stops third-party JavaScript from loading at all until the user clicks play. Use Lazy Load iFrames for non-video embeds (maps, social embeds) and the Video Facade for video.

YouTube / Vimeo Facade

Replaces YouTube and Vimeo iframes with a static thumbnail and play button. The real iframe is only loaded when the user clicks. Saves roughly 1.4MB of YouTube JavaScript on initial page load and prevents YouTube cookies being set until the user actually interacts with the video. Significant Lighthouse and Core Web Vitals improvement on pages with embedded videos.

How it works: the post-content filter scans for YouTube (`youtube.com/embed/...`, `youtube-nocookie.com/embed/...`) and Vimeo (`player.vimeo.com/video/...`) iframes and replaces each with a thumbnail-and-button facade. YouTube thumbnails are served from `i.ytimg.com`; Vimeo thumbnails are lazy-fetched via the public v2 API behind an `IntersectionObserver` so the network request only happens when the facade scrolls into view. Keyboard accessible (Enter / Space).

Lazy Load Background Images

CSS background images (specified via `background-image: url(...)` in a stylesheet) are not covered by native lazy loading. This option uses `IntersectionObserver` to apply the styles only when the container scrolls into view. Requires adding a specific class or data attribute to the containers — check the tooltip for the exact markup.

Lazy Threshold

How far outside the viewport an image can be before the browser starts loading it. The default (500 pixels below the fold) strikes a balance between bandwidth savings and avoiding blank spots when the user scrolls quickly. Reduce for mobile-heavy audiences on slow connections; increase for sites where users scroll fast.

Fade-in Animation

When enabled, lazy-loaded images fade in over a configurable duration rather than popping in abruptly. Cosmetic but pleasant. Add about 200ms of duration for a subtle effect without feeling slow.

Exclusion Options

Fine-grained controls for excluding specific images from lazy loading. Exclude by:

- **CSS selector:** Any valid selector, e.g. `.hero-image` or `#logo`
- **Class name:** Exclude by class (without the leading dot)
- **ID:** Exclude by element ID (without the hash)
- **Data attribute:** Exclude any element with a matching data attribute
- **Keyword in filename:** Substring match on the image URL
- **Parent container:** Exclude all images within a matching parent

Tip

Always exclude your LCP image from lazy loading. Lazy loading the Largest Contentful Paint image defeats the purpose — the whole point is that image should appear as early as possible.

DOM Monitoring

Images that are added to the page dynamically (by a carousel, infinite scroll, AJAX-loaded content, or a page builder's frontend editing) are not covered by a one-shot scan. DOM monitoring watches for new images being added and applies lazy loading to them too. Small performance cost on pages with lots of DOM mutations.

Database Tab

The Database tab handles housekeeping: pruning accumulated bloat, optimising tables, and keeping storage lean. Regular maintenance here keeps queries fast as your site ages.

Warning

Always take a full database backup before running any of the cleanup operations in this tab. The operations are permanent and cannot be undone. This is your first and last line of defence against mistakes.

Post Revisions

WordPress saves a copy of every draft as a post revision, forever by default. On busy editorial sites, this can double or triple the size of the `wp_posts` table. Two settings:

- **Keep Last X Revisions:** Limits the number of revisions stored per post going forward. 5 is usually enough. The scheduled cleanup will trim existing revisions to this number on each run.
- **Delete Existing Revisions:** A one-click button that prunes existing revisions to match the "Keep Last X" limit without waiting for the next scheduled run.

Auto-Drafts & Trash

WordPress creates an auto-draft every time you click Add New. Most of these are abandoned and never cleaned up. Similarly, trashed posts stay in the trash forever unless you explicitly empty it. Enable the auto-cleanup schedules to keep these table sections from growing. Each has its own configurable age threshold (default 7 days for drafts, 30 days for trash).

Spam & Unapproved Comments

Comments marked as spam or unapproved linger in the database. If you review and clear them manually, these toggles are not necessary. If you rely on Akismet to silently move spam to a pile and forget about it, enable automatic cleanup with the appropriate retention period (14 days for spam, 30 days for unapproved is reasonable).

Orphaned Metadata Scanners

Four separate scanners find metadata that references records which no longer exist:

- **Orphaned Post Meta:** `wp_postmeta` rows pointing to deleted posts
- **Orphaned Comment Meta:** `wp_commentmeta` rows pointing to deleted comments
- **Orphaned Taxonomy Relationships:** `wp_term_relationships` rows pointing to deleted objects
- **Orphaned Term Meta:** `wp_termmeta` rows pointing to deleted terms

Each scanner has a two-step workflow: Scan (count), then Delete (remove). Scanning is always safe. Deleting is permanent but usually very safe because the referenced records are already gone.

Transients

Transients are short-lived cached data in the `wp_options` table. Expired transients should be cleaned by WordPress automatically but often are not, and poorly written plugins can leave hundreds of thousands of them behind.

- **Get Transient Stats:** See how many transients exist, how many are expired, and total storage size.
- **Delete Expired Transients:** Removes only transients whose expiration has passed. Completely safe.
- **Delete All Transients:** Nuclear option. Removes every transient including active ones. Plugins that depend on transients will re-generate them on next use.
- **Auto-Delete Expired Transients:** Schedules expired-transient cleanup as part of the automatic database cleanup. Handles both regular and site-level transients (multisite-aware).

Cleanup Schedule

Three options: Daily (3 AM local time), Weekly (Sunday 3 AM local time), or Manual Only. Switching between options re-schedules the cron event automatically. The next-run time is shown on the tab.

Run Auto-Cleanup Now

Triggers the scheduled cleanup logic immediately. Useful for clearing a backlog or verifying that all your auto-cleanup toggles are doing what you expect before relying on the schedule. The Last Auto-Cleanup log table directly above the button refreshes after each run, showing how many items of each type were removed.

Table Operations

- **Optimise Tables:** Runs `OPTIMIZE TABLE` on every table. Reclaims space from deleted rows and defragments the table files. Safe but can be slow on large databases.
- **Convert MyISAM to InnoDB:** If any of your tables are still using the legacy MyISAM engine, this converts them to InnoDB. InnoDB offers row-level locking, transactions, and crash recovery.
- **Repair Tables:** Runs `REPAIR TABLE` on corrupted tables. Only relevant if you are actually seeing "table marked as crashed" errors.

Database Info

A diagnostic panel showing table sizes, row counts, engine types, and character sets. Useful for identifying which tables have grown unexpectedly large. For finding bloated autoloaded options specifically, use the Autoload Audit on the Diagnostics tab — it's purpose-built for that.

WebP / AVIF Tab

The WebP / AVIF tab handles two related but distinct jobs: converting your existing images to next-generation image formats for bandwidth savings, and managing image sizing and dimensions to fix common PageSpeed Insights warnings. The tab is split into four major sections.

Server Diagnostics

Before enabling any conversion, check the Server Diagnostics panel. It reports:

- GD library version (should be 2.0 or higher)
- WebP encoder support — must be present for WebP conversion (detected via `gd_info()['WebP Support']`)
- Upload folder write permissions (the converter writes WebP / AVIF files next to originals)
- .htaccess writability (only relevant if you plan to use the rewrite rules option)

Server AVIF capability

Before AVIF will produce any .avif files, the server needs a working AVIF encoder. Two paths are supported:

- **GD with libavif** — requires PHP 8.1+ *and* libgd compiled against libavif. The plugin detects this via `gd_info()['AVIF Support']`, which reflects what libgd was actually built with.
- **Imagick with libheif** — requires Imagick 7.0.25+ *and* ImageMagick built with libheif and a usable AVIF delegate.

Note

PHP 8.1+ declares `imageavif()` as a function regardless of whether libgd was built with AVIF support. Relying on `function_exists('imageavif')` is a false-positive trap on the very common shared-host configuration where the function symbol exists but calls fail silently. v1.13.7 onwards uses `gd_info()` instead — the same reliable pattern the plugin's WebP detection has always used.

The AVIF section of the WebP tab always shows the current capability status:

- If at least one path is supported, you see an info-style notice confirming AVIF encoding is available, with a quick GD / Imagick breakdown.
- If neither path is supported, you see a warning notice explaining the mismatch, with the same breakdown for context. The AVIF section's controls are hidden in this state.
- If AVIF is enabled in settings but the server can't actually encode AVIF, the plugin also shows a top-of-page admin notice on the plugin's own screens explaining the mismatch — so the situation is impossible to miss even if you don't visit the WebP tab directly.

WebP Conversion Settings

Automatic Conversion

When enabled, every new image uploaded to the Media Library is automatically converted to WebP in the background. Does not touch the original JPG or PNG — the WebP is created alongside it.

Compression Level

WebP quality from 1 (tiny, blurry) to 100 (identical to original). The default of 75 is a good compromise. Sites with a lot of photography may want 85–90; sites that prioritise speed above all can push down to 70.

HTML <picture> Tags

Wraps front-end images in <picture> elements with both WebP and original sources (and AVIF, if enabled). Works on any hosting environment and falls back to the original format for browsers without WebP / AVIF support.

.htaccess Rewrite Rules

Apache and LiteSpeed hosts can use URL rewriting to transparently serve WebP (and AVIF, if enabled) when the browser accepts it. Cleaner markup than the picture-tag approach but depends on a specific server environment. Not needed if you are already using the <picture> tag option.

AVIF Section

Two AVIF-specific controls. Both are hidden in the UI if the server lacks AVIF encoding capability.

Enable AVIF Conversion

On upload, generate an .avif sibling alongside the WebP version. The <picture> wrapper emits the AVIF source before the WebP source, so capable browsers (Chrome 85+, Firefox 93+, Safari 16.4+) use it preferentially. Requires Automatic Conversion and <picture> Tags to be enabled in the WebP section above for end-to-end delivery.

AVIF Quality

AVIF tolerates lower numbers than WebP for the same perceptual quality. The default of 60 is approximately equivalent to WebP at 75 — adjust within the 50–70 range depending on whether you are prioritising file size or fidelity.

Bulk Converter

To convert your existing Media Library, scroll to the Bulk Converter section. The section now has parallel **WebP Bulk Converter** and **AVIF Bulk Converter** subsections — each with its own Start / Clear History / Revert All buttons, progress bar, and status line.

Recommended order

Run them sequentially: WebP first to populate the WebP history, then AVIF to populate the AVIF history. After both have run, every existing image in your Media Library has both a .webp and .avif sibling, and the picture-tag wrapper serves the most efficient format the browser supports.

The AVIF Bulk Converter only renders when the server has a real AVIF encoder available (see capability check above). If you don't see the section, the WebP tab's AVIF diagnostic explains why.

Each converter shows live progress, skip reasons (usually "WebP would be larger than original"), and a running savings total. You can pause and resume at any time. On Media Libraries with thousands of images, expect the process to run for a while — let each one complete in one sitting if possible.

Conversion History

Updated in v1.13.8. Every WebP and AVIF file the plugin creates is recorded in its respective history option, and the Conversion History table merges both views into a single row per image. Columns: Original Image, Original Size, **WebP Size**, **AVIF Size**, Compression.

Each image appears as a single row with whichever format data exists — a dash shows where a format hasn't been generated for that image. The Compression column reports savings against whichever recorded format is smallest, so when AVIF is present it dominates (typically 20–30% smaller than WebP at equivalent perceived quality).

From the Bulk Actions dropdown above the table you can remove rows from the WebP history. Each subsection's Revert All button deletes that format's files from disk (originals remain) and clears that format's history.

Note

The plugin never modifies, moves, or deletes your original images. Every WebP and AVIF is a parallel file. Each Revert All only removes the plugin-created files for its format — your Media Library contents are always preserved.

Image Sizing & Dimensions

This section targets specific PageSpeed Insights warnings. The settings are independent from the WebP / AVIF conversion above — you can use any combination.

Resize Large Uploads

Any image uploaded larger than the configured maximum dimension (default 2560px on the longest edge) is automatically scaled down on upload using the WordPress core pipeline. Aspect ratio is preserved. The original file is never stored — the scaled version replaces it.

Add Missing Width & Height

Scans frontend HTML for `<img>` tags missing width or height attributes and adds them based on the actual image dimensions. Giving the browser the aspect ratio up front prevents Cumulative Layout Shift. Works with post content, Gutenberg blocks, Elementor widgets, attachment images, and post thumbnails.

Add `decoding="async"` to Images

Adds the `decoding="async"` attribute to all front-end `<img>` tags that don't already specify one. This lets the browser decode images off the main thread, improving INP on image-heavy pages. The LCP candidate (any image carrying `fetchpriority="high"`) is automatically skipped so it continues to decode synchronously.

Strip EXIF Metadata on Upload

On JPEG upload, removes EXIF, IPTC and XMP metadata (camera serial, GPS coordinates, embedded thumbnails). ICC colour profiles are preserved so colours stay accurate. Uses Imagick's `stripImage()` where available, falls back to GD re-encoding at quality 92. Privacy win — visitor photos no longer leak location data — plus typically a 5–30% file size reduction with no visible quality loss. Only affects new uploads; existing images are not touched.

Bulk Resize Tool

The Resize Large Uploads setting only affects images uploaded in the future. To downscale images that already exist in your Media Library, use the Bulk Resize tool at the bottom of this section.

Warning

The Bulk Resize tool permanently overwrites files on disk. Original files are not preserved. Always take a full file-system and database backup before running it. This is one of the two most destructive tools in the plugin (alongside Orphaned Media deletion).

The workflow is two-phase. Click Scan to see which images exceed the threshold. Review the list. Click Start Resize to begin. A progress bar, live log, and running savings total show what is happening. After resize, the plugin regenerates all registered sub-sizes using the WordPress core pipeline and cleans up any stale plugin-created WebP / AVIF files so they will be regenerated from the new dimensions.

Server Tab

Writes server-level configuration to `.htaccess` on Apache and LiteSpeed for browser caching and text compression. On Nginx and IIS, where `.htaccess` is ignored, the tab detects the server and shows an equivalent server-config snippet you can paste into your configuration manually.

Why this tab matters

Two of the most common PageSpeed Insights warnings — "Serve static assets with an efficient cache policy" and "Enable text compression" — are not actually code issues, they're server config issues. They can't be fixed by anything inside PHP. This tab writes the correct configuration for you so those warnings disappear without you needing to SSH in.

Server detection

At the top of the tab the plugin reports the detected web server (Apache, LiteSpeed, Nginx, IIS, or Other) based on the `SERVER_SOFTWARE` header. The `.htaccess` controls only do anything on Apache and LiteSpeed; on other servers an info notice explains the situation and the Nginx snippet section below provides a copy-ready equivalent.

Browser Cache Headers

Writes a `# MBR Browser Cache` marker block to `.htaccess` containing `mod_expires` and `mod_headers` rules. Expiry windows are tuned conservatively:

- 1 year for images (JPEG, PNG, GIF, WebP, AVIF, SVG), fonts (WOFF, WOFF2, TTF, OTF, EOT), and video (MP4, WebM) — files whose URLs are stable
- 1 month for CSS and JavaScript — usually fingerprinted via `?ver=` or hash, but the shorter window is a safety net
- 0 seconds for HTML — content needs to be fresh
- 1 hour for RSS / Atom feeds

A belt-and-braces `Cache-Control: public, max-age=31536000, immutable` header is also set on long-lived asset types so any proxy that ignores the `Expires` directive still respects the policy.

Brotli / Gzip Compression

Writes a `# MBR Compression` marker block. The configuration prefers Brotli (`mod_brotli`) where the server has it loaded and falls back to Gzip (`mod_deflate`) otherwise. Both filters are applied to HTML, plain text, XML, CSS, JavaScript, JSON, RSS, Atom, SVG, and WOFF / WOFF2.

Brotli typically saves another 15–20% over Gzip on text content. If both are loaded, Apache uses the most appropriate one based on the `Accept-Encoding` header sent by the browser.

On Nginx and IIS

Nginx and IIS don't read `.htaccess`, so the `.htaccess` writes are skipped. Instead, the tab displays an equivalent Nginx `server { }` block snippet that you can copy into your site config (typically at `/etc/nginx/sites-available/...` on Ubuntu / Debian, or `/etc/nginx/conf.d/` on other

distros). Reload Nginx after saving the config to apply.

Clean removal

Both marker blocks (`# MBR Browser Cache` and `# MBR Compression`) are removed cleanly from `.htaccess` when you disable the toggles or deactivate the plugin. The plugin only ever touches its own marker blocks; the rest of your `.htaccess` is left intact.

Third-Party Tab

Self-hosts common third-party tracking scripts (Google Analytics, Google Tag Manager, Facebook Pixel) and rewrites outbound `<script src=>` URLs in the page output to point at the local copies. Removes the PageSpeed Insights "Reduce the impact of third-party code" warning and stops first-paint network requests to `googletagmanager.com` and `connect.facebook.net`.

Why self-host tracking scripts

Even when tracking scripts themselves are well-written and lightweight, having to fetch them from third-party domains introduces extra DNS lookups, TLS handshakes, and round-trip latency. Self-hosting brings those costs down to zero. There's also a privacy / GDPR dimension: by serving the script bytes from your own domain, you avoid sending the visitor's IP to Google or Facebook just to download the script (the script itself still phones home once executed — this addresses asset delivery, not the tracking behaviour).

Supported scripts

- **Google Analytics (gtag.js)** — modern GA4 tag
- **Google Tag Manager (gtm.js)** — the container script (your tag IDs in the inline script are preserved unchanged)
- **Google Analytics (analytics.js)** — legacy Universal Analytics, retained for sites still using it
- **Facebook Pixel (fbevents.js)** — Meta tracking pixel

How it works

Each enabled script is downloaded once on enable, then refreshed daily by the `mbr_wp_performance_third_party_refresh` WP-Cron event. Files are stored under `/wp-content/uploads/mbr-wp-performance/third-party/`.

On every front-end page load, an output buffer rewrites the third-party `<script src=>` URLs to point at the local cached files. Query strings (e.g. `?id=GTM-XXXXX` on the GTM URL) are preserved intact, so your container IDs and tracking parameters continue to work exactly as before.

Refresh status

Beneath the toggles, the Cache Status section shows the time of the last refresh, per-script success / failure status, and a Refresh Now button. Run a manual refresh after enabling a script for the first time so you don't have to wait up to 24 hours for the daily cron to populate the cache.

What this doesn't change

Self-hosting addresses script-delivery costs but does not change the tracking behaviour itself. Once the (local) script executes, Google / Meta will still receive analytics events as they normally would. If your goal is GDPR compliance for the tracking itself rather than just script delivery, you also need a consent gate via something like the MBR Cookie Consent plugin.

Tip

Combine this with Delay JavaScript Until Interaction on the JavaScript tab for the strongest result: the script is served from your own domain (no third-party fetch on initial load) and only executes when the user actually interacts with the page (no third-party JS work on initial load either).

Diagnostics Tab

Three diagnostic tools in one place: the Autoloaded Options Audit, the WP-Cron Viewer, and the Caching Plugin Conflict Detector. None of these are theoretical — they target the most common WordPress performance footguns.

Caching Plugin Conflicts

At the top of the tab, the conflict detector scans for the presence of known caching plugins and shows which MBR WP Performance options overlap with each. The most common pitfall when stacking performance plugins is having Defer JavaScript or Minify CSS or browser-cache headers enabled in two places at once — usually results in broken JS, no styling, or a PageSpeed score that won't improve.

Currently detected:

- **WP Rocket** — overlaps on Load CSS Asynchronously, Minify CSS / JS, Remove Unused CSS, Defer / Delay JavaScript, LazyLoad for iframes / videos, Browser Cache, GZIP
- **W3 Total Cache** — overlaps on Minify CSS / JS, Defer / Async JavaScript, Browser Cache, HTTP compression
- **LiteSpeed Cache** — overlaps on CSS Minify / Async / Critical, JS Defer / Delay / Minify, WebP Replacement, Browser Cache
- **FlyingPress** — overlaps on Async CSS, Critical CSS, Defer / Delay JS, YouTube placeholder
- **WP Super Cache** — overlaps on Compress pages
- **Perfmatters** — overlaps on Defer / Delay JavaScript, Remove Unused CSS, Disable jQuery, Remove query strings
- **Autoptimize** — overlaps on Minify CSS / JS, Inline and Defer CSS

For each active plugin the detector lists exactly which MBR options overlap. Pick one or the other — never enable the same optimisation in both plugins.

Autoloaded Options Audit

Every option in the `wp_options` table marked `autoload=yes` is loaded on every page hit. A handful of core options need this, but plugins frequently mark their large data blobs (cached API responses, settings serialisations, log dumps) as autoloaded by mistake — and that data is then read out of the database on every visitor request. This is consistently the single most common WordPress database performance killer.

The audit panel shows the total bytes currently being autoloaded across how many options, plus the top 30 by size. Each row has a one-click "Disable autoload" button. Protected WordPress core options (`siteurl`, `home`, `active_plugins`, `template`, `stylesheet`, and around 85 others) cannot be modified — the toggle is replaced with an em-dash in the action column. Transients (autoloaded transients are almost always a bug) are flagged in the Notes column.

Since v1.13.3, the plugin's own options are excluded from the audit list — matching the function's docblock, which had previously diverged from the SQL. You can't disable autoload on this plugin's own configuration via its own tool, which would be a small but real footgun.

Tip

If total autoloaded data is over 1MB, you almost certainly have one or two badly-behaved plugins polluting the options table. Sort by size, work through the top 5 rows, and disable autoload on anything that looks like "plugin name + huge serialised blob". If you disable autoload on something that turns out to be needed, the data is still in the database — re-enable autoload from the same panel and the plugin works again.

WP-Cron Viewer

Lists every scheduled cron event with its hook, next-run time, recurrence interval, and a Callback? column showing whether any PHP callback is currently registered for the hook. Events with no callback are flagged "orphan" and can be unscheduled with one click — these are almost always left behind by plugins that have been deactivated without cleaning up their scheduled events.

Real cron vs. WP-Cron

If `DISABLE_WP_CRON` is defined as `true` in your `wp-config.php`, the tab confirms that and steps out of the way. Otherwise, the tab explains how to switch to a real system cron job and shows the two snippets you need: the `wp-config.php` line plus a crontab entry that hits `wp-cron.php` every 5 minutes.

On low-traffic sites, WP-Cron is unreliable because it only fires when someone visits the site. Real cron eliminates that variance and makes scheduled events (including the plugin's own database cleanup and third-party-script refresh) reliable.

Orphaned Media Tab

The Orphaned Media tab finds media attachments in your Media Library that are no longer referenced anywhere on your site, and provides a safe two-stage workflow for removing them — with a configurable restore window so the database record can be reinstated if you change your mind. Coverage spans images, video, audio, documents, and archives.

Warning

This is one of the two most destructive tools in the plugin (alongside the Bulk Resize tool on the WebP / AVIF tab). Deletion physically removes the original file, every WordPress sub-size variant, and any matching .webp / .avif sibling files. The database record is preserved in a staging table for the configured restore window, but the file bytes themselves are gone. Always test on a staging copy first and take a full file-system backup before running deletions on a live site.

How detection works

For every eligible attachment in your Media Library, the scanner runs four checks and only flags an attachment as a candidate if none of the definitive checks find a reference:

- **Post parent:** Is the attachment's `post_parent` a post that still exists and is not in the trash?
- **Featured image:** Is the attachment ID stored as `_thumbnail_id` on any post?
- **Post content:** Is the attachment referenced inside any non-trashed post's `post_content`? The scanner matches by attachment ID (Gutenberg `wp-image-{ID}` class, gallery shortcode references) and by filename stem so sized variants are caught alongside the original.
- **Postmeta string-search:** Does the attachment ID appear as a value or substring in any postmeta row (excluding `_thumbnail_id`)? Coarser check that picks up references in serialised data — ACF fields, page builder data, custom field plugins.

A match in any of the first three checks is treated as definitive and the attachment is not flagged. A match only in the postmeta string-search is treated as suggestive but not authoritative — those attachments are flagged with Review confidence. Attachments with no matches at all are flagged with High confidence.

Confidence tiers

- **High:** No references found in any check. Eligible for bulk-delete. Safest candidates to remove.
- **Review:** The postmeta string-search returned a match but the `post_parent`, featured-image, and `post_content` checks all came back clean. Must be deleted individually after manual inspection.

Media types

Media-type checkboxes on the tab control which categories of attachment the scanner considers: Images, Videos, Audio, Documents, Archives. Defaults to images-only to preserve v1.10.0-equivalent behaviour on upgrade. Per-type stat breakdown above the candidate table shows the count and reclaimable bytes for each enabled type.

Settings

Restore Window

Controls how long deleted attachment records remain in the staging table before being permanently purged by the daily cron. Options are 7, 14, 30, or 60 days, or "Keep forever" if you would rather purge manually.

Exclusions

A list of attachment IDs that should never be flagged as orphan, even if they appear unused. Use this for attachments you know are referenced from somewhere the scanner cannot see — a custom theme template, an external embed, content scheduled for the future. You can also add an exclusion directly from the candidate list with the Keep button next to any row.

Running a scan

Click Run Scan to start. The scanner processes attachments in batches of 50 via background AJAX requests with a progress bar. On a Media Library of around 1,000 images expect 30 seconds to a minute. Larger libraries take proportionally longer; the postmeta string-search is the slowest individual operation and dominates total scan time on sites with very large wp_postmeta tables.

Each scan wipes the previous candidate list and rebuilds from scratch. Running a fresh scan after editing content is a good habit.

The candidate list

Three stat cards summarise the results:

- **High Confidence:** Count and total disk size of high-confidence orphans.
- **Review Required:** Count and total disk size of review-tier candidates.
- **Total Reclaimable:** Combined disk size you would free by deleting every candidate.

The table itself shows a thumbnail preview (for images) or type icon (for non-images), the filename, the attachment ID, the file URL, the file size, the confidence badge, and per-row Delete and Keep actions. Type, Sort, and Confidence filters above the table let you narrow the view.

Deleting orphans

Single deletion

Click Delete on any row. The plugin re-verifies orphan status one more time at delete time (so any references added since the scan are caught), captures the full attachment post row, postmeta, and a manifest of every file about to be removed, then physically deletes the original file, the scaled variant, every sub-size, and matching .webp / .avif siblings. The WordPress attachment record is then removed via `wp_delete_attachment()`.

Bulk deletion

Tick the checkboxes next to one or more high-confidence rows and click Delete Selected. Review-tier rows have their checkboxes disabled — bulk deletion is restricted to high confidence by design.

Pre-deletion re-verification

Between scan and delete, you might publish a new post that references one of the candidates. Every deletion runs an abbreviated re-check immediately before destroying the file. If the attachment is now

referenced anywhere, the deletion is blocked and the UI reports the situation.

Recently Deleted (the restore queue)

Beneath the candidate list, the Recently Deleted section shows every staged deletion still within its restore window. Each row has a Restore Record button.

Warning

Restore is record-only. The image file bytes are physically deleted at the moment of staging, not at the end of the restore window — so by the time you click Restore Record, the original file is gone. The restored attachment will appear in the Media Library with a broken thumbnail and will need to be re-uploaded for the actual image to appear. The restore is useful when a deletion has left content with a broken reference and you want the database record back.

Daily cron purge

A daily WP-Cron event called `mbr_wp_performance_orphan_purge` removes any staging records whose purge date has passed. Records configured with the "Keep forever" restore window are never purged automatically.

A safe first scan

- Take a file-system backup of `/wp-content/uploads/` and a database backup. Non-negotiable.
- Set the restore window to 30 days (default) or 60 days for extra room.
- Run a scan and let it complete fully.
- Sort the candidate list by file size (largest first) — these are the highest-value targets.
- For the top 5–10 high-confidence rows, open the Media Library edit screen and visually confirm the file is one you do not recognise as currently in use.
- Delete one or two individually, navigate around the front end, and confirm nothing has broken. Only then move on to bulk-delete.
- Leave Review-tier rows alone on the first pass. Come back to them once you have a feel for what the scanner is flagging and why.

WooCommerce Tab

The WooCommerce tab only activates when WooCommerce is installed and active. It consolidates the WooCommerce-specific optimisations that used to be scattered across the Core and CSS tabs, and adds a set of deeper controls that target WooCommerce's most common performance drains.

Geolocation Advisory

At the top of the tab, the plugin checks your WooCommerce geolocation setting and displays a notice if it is configured in a way that interacts badly with full-page caching.

- **Geolocate:** WooCommerce performs a server-side IP lookup on every page load. Full-page caches are effectively disabled because every response varies per visitor.
- **Geolocate (with page cache support):** WooCommerce appends a `?v=<timestamp>` query string to URLs. Depending on your cache plugin's configuration, this can either bypass the cache entirely or create a very large number of cache variants.

The advisory notice includes a direct link to WooCommerce's General settings where you can change the setting to "Shop base address" (no geolocation), which is cache-friendly.

Frontend Assets

Cart Fragments

WooCommerce fires an admin-ajax request called `wc-cart-fragments` on every single page load to update the mini-cart display. On cached sites, this uncached request is often the single biggest source of Time to First Byte latency. Three options:

- **Default (WooCommerce behaviour):** Fragments run on every page. Mini-cart updates live.
- **Disable on non-shop pages:** Fragments run on shop, cart, checkout, product, and account pages only. Mini-cart may be stale elsewhere until page reload.
- **Disable site-wide:** Fragments never run. Mini-cart updates only on page reload. Biggest performance win.

Tip

Most stores can safely disable cart fragments site-wide. The trade-off is that if your theme has a persistent header with a cart count, the count will not update live after an Add to Cart — the user has to refresh to see the new count. If that is acceptable, the performance win is substantial.

Disable WC Scripts / Styles / Block Assets on Non-Shop Pages

Three related toggles that dequeue WooCommerce's scripts, stylesheets, and block editor assets on any page that is not a shop, product, cart, checkout, or account page. Big win on content-heavy sites where most pages have nothing to do with the shop.

Disable Password Strength Meter

WooCommerce includes the `zxcvbn` password strength library for the account registration flow. It is loaded on every frontend page in many themes. Disabling saves around 50KB on non-registration pages.

Admin Options

Disable Marketplace Suggestions

Stops WooCommerce fetching extension recommendations from WooCommerce.com and showing them inside your admin. Removes outbound API calls and declutters the admin experience.

Disable Dashboard Widgets

Removes the WooCommerce Status, Recent Reviews, and setup widgets from the WordPress dashboard. Fewer database queries on dashboard load.

Disable WC Admin Scripts on Non-WC Pages

WooCommerce loads its heavy wc-admin React bundles on many non-WooCommerce admin screens. Enabling this dequeues them where they are not needed. Makes the regular wp-admin noticeably snappier on stores that also use heavy-admin plugins like Yoast or ACF.

Database Cleanup

Enable Weekly Automated Cleanup

When enabled, the plugin's weekly cron runs three maintenance jobs automatically: expired sessions, WooCommerce transients, and Action Scheduler history cleanup. The tab shows the next scheduled run time and a last-run log.

Action Scheduler Retention

WooCommerce's Action Scheduler keeps completed and failed actions in the wp_actionscheduler_actions table for 30 days by default. On busy stores — especially those running WooCommerce Subscriptions — this table can grow to hundreds of thousands of rows, slowing admin queries noticeably. Options for 14, 7, and 3 day retention. The retention filter only affects future cleanup runs; use Run Cleanup Now to drain an existing backlog.

Expired Sessions

Shows a count of expired rows in the wp_woocommerce_sessions table and a button to delete them. WooCommerce's own cleanup cron often fails on sites with flaky WP-Cron, so this is a useful manual lever.

WooCommerce Transients

One-click cleanup of product, shop order, and expired transients using WooCommerce's own helper functions.

Multisite Network Support

The plugin has full WordPress Multisite compatibility. Network-activate it and manage default settings from the Network Admin, with per-site override control for situations where individual sites need something different.

Network Admin settings

Go to Network Admin → Settings → WP Performance. You will see the same tab structure as the regular admin, but the settings you save here become the network defaults for every site.

Push to all sites

After configuring your network defaults, click Push Settings to All Sites to copy them over to every site in the network. You can also select specific sites. Existing site-level settings are overwritten by this operation.

Import from a site

Already have one site configured the way you want? Import its settings as the network default with a single click. Useful when migrating from per-site configuration to network-wide consistency.

Per-site override control

A master toggle in the Network Admin determines whether individual site admins can change their settings or whether they are locked to the network defaults. Super admins can always change settings regardless of this toggle.

New sites

Sites created after the plugin has been network-activated automatically inherit the network defaults on creation. No manual setup per site.

Common Workflows

A set of recipes for common goals. Each assumes you have taken backups and are testing changes on a staging environment first.

"I want to improve my Google PageSpeed Insights score"

- **Diagnostics tab:** Check the Caching Plugin Conflicts panel first. If another caching plugin is active, decide who owns each optimisation before enabling anything below.
- **Server tab:** Enable Browser Cache Headers and Brotli / Gzip Compression. These alone clear two PSI warnings on most sites.
- **Lazy Loading tab:** Enable image and iFrame lazy loading. Enable the YouTube / Vimeo Facade. Set an exclusion for your LCP image.
- **Preloading tab:** Enable Auto-prioritise First Image. Add your LCP image URL to Preload Critical Images. Enable Hover Prefetch.
- **CSS tab:** Enable Minify CSS and Load CSS Asynchronously. Generate critical CSS using a viewport-aware tool (Penthouse, Critical, Critters, or an online generator) and paste it into the Critical CSS Code textarea. Enable Inline Critical CSS. If you skip the critical-CSS step, the safety interlock will keep the first two stylesheets render-blocking so you still get a real first paint — partial benefit, no FOUC.
- **JavaScript tab:** Enable Defer JavaScript and Move Scripts to Footer. Add analytics scripts to the Delay Until Interaction list.
- **Third-Party tab:** Enable self-hosting for whichever tracking scripts you use (GA, GTM, Pixel). Click Refresh Now to populate the cache.
- **Fonts tab:** Enable Self-Host Google Fonts and click Download Fonts. Set font-display to swap.
- **WebP / AVIF tab:** Run the WebP bulk converter, then (if AVIF is supported) the AVIF bulk converter. Enable Resize Large Uploads, Add Missing Width & Height, decoding="async", and Strip EXIF Metadata on Upload.
- **Core tab:** Enable Minify HTML. If you're on WordPress 7.0 with no plans to wire AI providers, also enable Disable AI Features.

"My WooCommerce store feels sluggish"

- **WooCommerce tab:** Set cart fragments to Disable site-wide (or non-shop pages if you rely on the live mini-cart).
- Enable Disable WC Scripts on Non-Shop Pages and Disable WC Styles on Non-Shop Pages.
- Enable Disable WC Admin Scripts on Non-WC Pages to speed up the regular wp-admin.
- Set Action Scheduler retention to 14 or 7 days.
- Enable Weekly Automated Cleanup and click Run Cleanup Now once to drain any backlog.
- Check the geolocation advisory — if shown, switch WooCommerce to Shop base address.
- **Diagnostics tab:** Check the Autoloaded Options Audit. WooCommerce stores often accumulate large autoloaded options from analytics extensions and admin notices.

"I need to meet GDPR requirements on third-party assets"

- **Fonts tab:** Enable Self-Host Google Fonts and click Download Fonts. Stops sending visitor IPs to Google's font servers.
- **Third-Party tab:** Enable self-hosting for your GA / GTM / Pixel script. Stops the first-paint script fetch from going to googletagmanager.com or connect.facebook.net.
- **Lazy Loading tab:** Enable YouTube / Vimeo Facade — stops YouTube cookies being set on first paint.
- Verify in your browser's Network tab that no first-paint requests go to fonts.googleapis.com, fonts.gstatic.com, googletagmanager.com, or connect.facebook.net.
- Note that this only addresses asset delivery. For consent-gated tracking behaviour you also need a cookie consent banner like MBR Cookie Consent.

"My site has ballooned and I want to clean up the database"

- **Database tab:** Take a full backup first. Always. Set Keep Last X Revisions to 5 and click Delete Existing Revisions. Run all four orphaned metadata scanners. Click Get Transient Stats then Delete Expired Transients. Click Optimise Tables. Check Database Info before and after to see the savings.
- **Diagnostics tab:** Open the Autoloaded Options Audit. Disable autoload on any non-protected rows over a few hundred KB. Open the WP-Cron Viewer and unschedule any orphan events.
- **Orphaned Media tab:** If you suspect unused media is contributing to the bloat, follow the safe-first-scan checklist in the Orphaned Media chapter.

"I want to delay analytics without breaking them"

- **JavaScript tab:** Enable Delay JavaScript Until Interaction.
- Add these identifiers to the delay list (one per line): google-analytics, gtag, googletagmanager, fbq, fbevents, hotjar, clarity, intercom.
- Set the timeout (in seconds) to your taste — 3 seconds is a safe default that catches most bounced sessions.
- Test your site — scripts should only fire after first scroll, click, or keypress, or after the timeout if neither happens.
- Verify in your analytics dashboard that events still record (with a small delay).

"I want to reclaim disk space from unused images"

- Backups first: file-system backup of /wp-content/uploads/ and a database backup. The most destructive workflow in this section.
- **Orphaned Media tab:** Set the restore window to 30 or 60 days.
- Click Run Scan and let it complete.
- Sort the candidate list by file size, largest first. Spot-check the top few rows.
- Delete one or two individually first. Navigate the front end and confirm nothing has broken.
- Once confident, tick the high-confidence rows in batches and use Delete Selected. Leave Review-tier rows alone on the first pass.
- Wait a day or two before deleting Review-tier rows individually — gives you time to spot any subtle breakages from the high-confidence pass.

- Keep an eye on Recently Deleted for the duration of your restore window in case a missed reference surfaces.

Troubleshooting

A setting broke my site

The fastest recovery: open a private browsing window, log into wp-admin, open the plugin settings, and disable the setting you most recently enabled. Save. If you cannot access wp-admin, rename the plugin folder via SFTP — that deactivates the plugin entirely. Settings are preserved for when you reactivate.

After enabling "Remove jQuery", something broke

jQuery is a sprawling dependency. Disable Remove jQuery first and test whether the site recovers. If it does, your theme or a plugin depends on jQuery — do not enable the option. There is no reliable shortlist of what works and what does not. Use the test mode toggle to scope the removal to logged-out visitors only while you investigate.

Two performance plugins are fighting each other

Open the Diagnostics tab. The Caching Plugin Conflicts panel detects known caching plugins and shows which MBR options overlap with each. Pick one or the other — disabling overlapping options in MBR WP Performance and leaving them on in your caching plugin (or vice versa) is usually the right move.

Fonts look different after enabling self-host

The most common cause is that the initial scan missed a font that is declared somewhere unusual. Click Clear Font Cache in the Fonts tab, then add the missing font families to the Manual Fonts field and click Download Manual Fonts.

PageSpeed Insights still reports "Serve static assets with an efficient cache policy"

Open the Server tab. Confirm Browser Cache Headers is enabled and the detected server is Apache or LiteSpeed (the .htaccess writes do nothing on Nginx and IIS — copy the Nginx snippet shown on the tab instead). If you are behind a CDN like Cloudflare, also check that the CDN is not overriding your origin's cache headers.

PageSpeed Insights still reports "Image elements do not have explicit width and height"

Ensure Add Missing Width & Height is enabled in the WebP / AVIF tab. Clear any page caches. The feature works at runtime on the frontend HTML, so cached versions of pages will still show the old markup until the cache is flushed.

I see a flash of unstyled content (FOUC) after enabling Async CSS

v1.13.4 added a safety interlock that prevents this: when Async CSS is on but Inline Critical CSS is off (or the Critical CSS Code textarea is empty), the first two stylesheets stay render-blocking so the page still paints correctly. If you are still seeing FOUC, either upgrade to v1.13.4 or later, or paste critical CSS from a viewport-aware tool (Penthouse, Critical, Critters) into the Critical CSS Code textarea and enable Inline Critical CSS.

Where did the "Auto-Generate Critical CSS" button go?

Removed in v1.13.4. The button used regex-based extraction across hardcoded selectors, which is structurally too crude to produce safe inline CSS for a modern WordPress site — it stripped @media wrappers off rules, matched anything starting with a single-letter target, ignored CSS variables on :root, and indiscriminately inlined every @font-face declaration. The Critical CSS Code textarea

remains — paste critical CSS produced by a proper viewport-aware tool (Penthouse, Critical, Critters, or any of the online critical-CSS generators) and enable Inline Critical CSS.

AVIF is enabled but no .avif files are being created

This was the symptom of the v1.13.7 false-positive detection bug. The previous `function_exists('imageavif')` check returned true on PHP 8.1+ regardless of whether libgd was actually compiled against libavif. On the common shared-host configuration where libavif wasn't linked, the function symbol existed but calls failed silently.

If you are running v1.13.7 or later, the WebP / AVIF tab now shows a clear diagnostic notice with the GD AVIF and Imagick AVIF capability indicators. If both are **X**, the toggle is hidden and AVIF conversion is silently skipped — ask your host to enable libavif (for GD) or libheif (for Imagick), or accept that AVIF isn't available on your host. WebP-only is still a substantial win on its own.

Plugin updates do not seem to apply on SiteGround / other OPcache hosts

SiteGround (and several other shared hosts) ship PHP with OPcache enabled and `opcache.validate_timestamps=0`, which means PHP files updated on disk are not re-read into the bytecode cache automatically. If a plugin update appears to be missing — for instance, a setting page that should look different still looks the same after upgrading — clear OPcache manually.

On SiteGround: Site Tools → Speed → Caching → System tab → Flush Cache. On other hosts, see your control panel's cache section. As a last resort, you can drop a one-shot file at `wp-content/plugins/mbr-wp-performance/_opcache_reset.php` containing `<?php opcache_reset(); echo 'reset'; ?>`, visit it once in the browser, then delete it. Some hosts disable `opcache_reset()` for non-CLI; if so, the host's control panel route is the only path.

Third-party scripts are not being self-hosted

Open the Third-Party tab and check the Cache Status panel. If no refresh has run yet, click Refresh Now to populate the cache immediately. If refreshes are failing with HTTP errors, your server may be blocking outbound requests — check with your host. The script is rewritten only when a local cached copy exists, so without a successful refresh the original third-party URL continues to be served.

WooCommerce geolocation notice keeps appearing

The notice only appears when WooCommerce's default customer location is set to Geolocate or Geolocate (with page cache support). Go to WooCommerce → Settings → General and change Default customer location to Shop base address. Save.

The weekly cleanup does not appear to be running

WordPress's cron is not a real cron — it fires when someone visits the site. On low-traffic sites, scheduled events can miss their window. Check the next-run line in the Database tab; if it is in the past, your server-side cron is unhealthy.

Open the Diagnostics tab → WP-Cron Viewer for guidance on setting up a real system cron job. Two fixes: either set up a real cron job calling `wp-cron.php` every 5 minutes, or use WP-CLI to trigger manually with `wp cron event run mbr_wp_performance_database_cleanup`.

The orphan scan is taking a very long time

The slowest part of the scan is the postmeta string-search, which runs one LIKE query per attachment against the entire `wp_postmeta` table. On sites with very large postmeta tables — typically caused by aggressive page builders, complex ACF configurations, or WooCommerce stores with thousands of

products — this can turn a 30-second scan into several minutes. Be patient; the scanner is batched to avoid timeouts.

An image I know is in use was flagged as orphan

Most likely the image is referenced in a page builder data store the scanner cannot fully see — Elementor's `_elementor_data`, Bricks content, Beaver Builder layouts, Oxygen components. Click the Keep button next to the row to add it to the exclusions list permanently.

Inline status messages on the Database panel look low-contrast

This was fixed in v1.13.6. If you are still seeing near-black text on the plugin's green / red / amber notice backgrounds, you are on an older release. Upgrade to v1.13.6 or later (and clear OPcache and the browser cache after upgrading).

The plugin settings page will not load

Very rare, but usually caused by a conflict with another performance plugin. Open the Diagnostics tab's Caching Plugin Conflicts panel — if you have a caching plugin installed, that's the first suspect. Otherwise deactivate other performance plugins temporarily.

Frequently Asked Questions

Will this plugin break my site?

It is designed to be safe, but performance optimisations always carry some risk because they interact with themes and other plugins in complex ways. Always take a backup, test on staging, and enable features one at a time. Features are independently toggleable so you can always reverse a specific change.

Can I use this alongside a caching plugin?

Yes — the Diagnostics tab includes a conflict detector that specifically helps with this. The plugin provides complementary optimisations that work with any caching solution. There is deliberately no overlap with full-page caching; this plugin does not try to replace your cache layer. Open the Diagnostics tab → Caching Plugin Conflicts for guidance on which MBR options to disable when a caching plugin is active.

Is this plugin GDPR compliant?

The plugin itself does not collect, transmit, or store any visitor data. Features like self-hosted Google Fonts (Fonts tab), self-hosted third-party scripts (Third-Party tab), and the YouTube / Vimeo Facade (Lazy Loading tab) actively improve GDPR compliance by preventing visitor data from being sent to third parties on first paint. No tracking, no phoning home, no external calls.

What is the difference between WebP and AVIF?

Both are next-generation image formats designed to replace JPEG and PNG. AVIF (released 2019) is typically 20–30% smaller than WebP (released 2010) at equivalent perceived quality, but requires more recent browser support (Chrome 85+, Firefox 93+, Safari 16.4+). Server-side encoding requirements are also stricter (PHP 8.1+ with libavif, or recent Imagick with libheif). Enable both: the `<picture>` wrapper sends AVIF to capable browsers, WebP to the next tier, and the original format as fallback.

Does AVIF need anything special on the server?

Yes — server-side AVIF encoding needs PHP 8.1+ compiled with libavif (for the GD path) or Imagick 7.0.25+ with libheif. The WebP / AVIF tab includes a capability check showing whether each is available; the AVIF section is hidden in the UI if neither is present. A common gotcha: `function_exists('imageavif')` returns true on PHP 8.1+ even when libgd was built *without* libavif, so historic detection was unreliable. v1.13.7 onwards uses `gd_info()['AVIF Support']` instead, which reflects what libgd was actually compiled with. If your site previously had AVIF "enabled" but no `.avif` files appearing, you were almost certainly hitting that false positive.

Does the WebP / AVIF converter modify my original images?

No. WebP and AVIF files are always created alongside the originals with the same filename and a different extension. The originals are never touched. Deactivating the plugin or clicking Revert All removes the plugin-created files; the originals remain.

What happens if I deactivate the plugin?

All optimisations are immediately disabled. Settings are preserved in case you reactivate later. WebP and AVIF files tracked in the registries are cleaned up automatically. All `.htaccess` marker blocks (MBR WebP, MBR AVIF, MBR Browser Cache, MBR Compression) are removed. The third-party script refresh cron is unscheduled. Note that the Bulk Resize tool is destructive — resized files are permanent and are not restored on deactivation. The Orphaned Media feature is similarly destructive: deleted files

cannot be recovered after deactivation.

How do I provide critical CSS now that the Auto-Generate button is gone?

Use a viewport-aware tool that renders the page in a headless browser and extracts only the rules whose selectors match elements above the fold. Good options include Penthouse (CLI / Node), Critical (CLI / Node), Critters (used inside a build pipeline), or any of the various online critical-CSS generators that take a URL and return CSS. Paste the output into the Critical CSS Code textarea on the CSS tab, enable Inline Critical CSS, and save. Regenerate after major design changes.

Why did my CSS toggles seem to keep resetting on earlier versions?

Three font-related fields on the Fonts tab had their HTML form inputs scoped to `[css][...]` instead of `[fonts][...]`. Saving the Fonts tab therefore POSTed a partial CSS section, and the CSS sanitiser replaced the whole stored CSS section with the partial input — wiping every other CSS toggle. Fixed in v1.13.5. If you're running v1.13.5 or later, saving the Fonts tab no longer touches your CSS settings.

Is it safe to bulk-delete Review-tier orphans?

No, and the plugin will not let you. Bulk deletion is restricted to high-confidence rows (no references found in any check). Review-tier rows must be deleted individually after manual inspection. The reason is that Review-tier matches come from the postmeta string-search, which is a coarser check that can miss meaningful references inside serialised data — particularly inside page builder layouts.

Can I undo a deletion after the restore window has passed?

No. Once the daily cron has purged a staging row, the database record of that deletion is gone — there is nothing left to restore. Even within the restore window, only the database record can be reinstated, not the image file itself (file bytes are deleted at the moment of staging). For true file-level recovery, take a file-system backup of `/wp-content/uploads/` before running deletions.

Does the plugin work with Elementor / Beaver Builder / Divi / Oxygen / Bricks?

Yes. The plugin automatically disables all optimisations inside page builder editors to prevent conflicts with the editing experience. On the frontend, all optimisations apply normally. No configuration needed — detection is automatic. Upload-pipeline modules (WebP, AVIF, EXIF strip, resize) run regardless of editor context, so images uploaded through builder media pickers get the same treatment as direct Media Library uploads. Note that the Orphaned Media scanner does not yet directly inspect builder-specific data stores, so always run it on a staging copy first if you rely heavily on a page builder.

Why does Combine JavaScript / Combine CSS / Remove Unused CSS do nothing?

These three toggles are preserved in the UI for forward compatibility but currently act as no-ops with a clear admin notice. A safe implementation of each requires substantial engineering work that is scheduled for a future release. In the meantime, use Defer, Delay, Inline Critical CSS, and Async CSS for similar real-world wins, and use the MBR Advanced Asset Manager plugin for per-URL stylesheet control.

Does the plugin slow down the wp-admin?

No — most optimisations apply only to the frontend. The admin-side features (like Disable WC Admin Scripts on Non-WC Pages on the WooCommerce tab) actively make wp-admin faster. The Diagnostics tab panels do execute database queries (the Autoload Audit reads `wp_options`; the Cron Viewer reads the cron array) but only when you visit that tab.

How do I update the plugin?

Download the latest zip from littlewebshack.com, then in wp-admin go to Plugins → Add New → Upload Plugin. Upload the new zip and WordPress will offer to replace the existing installation. Settings are preserved through updates; migrations run automatically. If you're on SiteGround or another OPcache-heavy host and the new version doesn't seem to take effect, clear OPcache — see the Troubleshooting chapter for steps.

Where can I report bugs or request features?

GitHub Issues at github.com/harbourbob/mbr-wp-performance is the fastest route. You can also email via madebyrobert.co.uk or leave a message via littlewebshack.com.

Is there a Pro version?

No. There are no premium tiers, no paid upgrades, and no feature gating. Everything in the plugin is available for free. If you find it useful, a contribution via buymeacoffee.com/robertpalmer is always appreciated.

Resources & Support

Where to get help

- **Website:** littlewebshack.com — plugin documentation, downloads, blog posts
- **Author:** madebyrobert.co.uk — agency site and contact form
- **GitHub:** github.com/harbourbob/mbr-wp-performance — issues and feature requests
- **Buy me a coffee:** buymeacoffee.com/robertpalmer — tip jar if the plugin has saved you time

Related plugins in the portfolio

MBR WP Performance is one of a family of free plugins from Made by Robert. Others worth knowing about:

- **MBR Advanced Asset Manager:** Conditional dequeuing of scripts and styles on specific URLs — complements this plugin's more blanket optimisations, and the right tool for per-page asset removal that Remove Unused CSS does not yet do here.
- **MBR Cookie Consent:** Privacy-first cookie consent banner with IAB TCF v2.3, Google Consent Mode v2, and GPC signal support — pairs well with this plugin's Third-Party tab.
- **MBR Theme Generator:** Build a custom WordPress theme — including child themes for both classic and FSE / block parents — from a guided UI.
- **MBR Intelligent Site Assistant:** AI-powered on-site help with configurable intents, synonyms, and appearance.
- **MBR Bulk WP Detector:** Scans URLs to detect whether they are running WordPress and harvests page-level intel.

Philosophy

Every plugin in the Made by Robert portfolio is free, with no premium tiers, no upsells, and no tracking. The goal is to make the kind of tools site owners actually need available without friction. If you find the plugin useful, consider leaving a thumbs-up on its GitHub repository or sharing it with someone who might benefit.

Thanks for using MBR WP Performance. Happy optimising.